



UNIVERSIDAD DE GRANADA

Periféricos y Dispositivos de Interacción Humana

Práctica 6: Reconocimiento de Imágenes

Pablo Soriano Fleitas

Ejercicio 1: Caras de personas en imágenes

Para este primer ejercicio debo de reconocer caras de personas en imágenes, marcando las zonas de la imagen que representen los rostros de las personas que aparecen en ellas.

Código:

```
import cv2

#Cargamos nuestro modelo clasificador de Haar:
modelo = cv2.CascadeClassifier('./modelos\haarcascade_frontalface_alt.xml')

#Cargamos la imagen
img = cv2.imread('./imagenes\imagen_caras.jpg')
imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

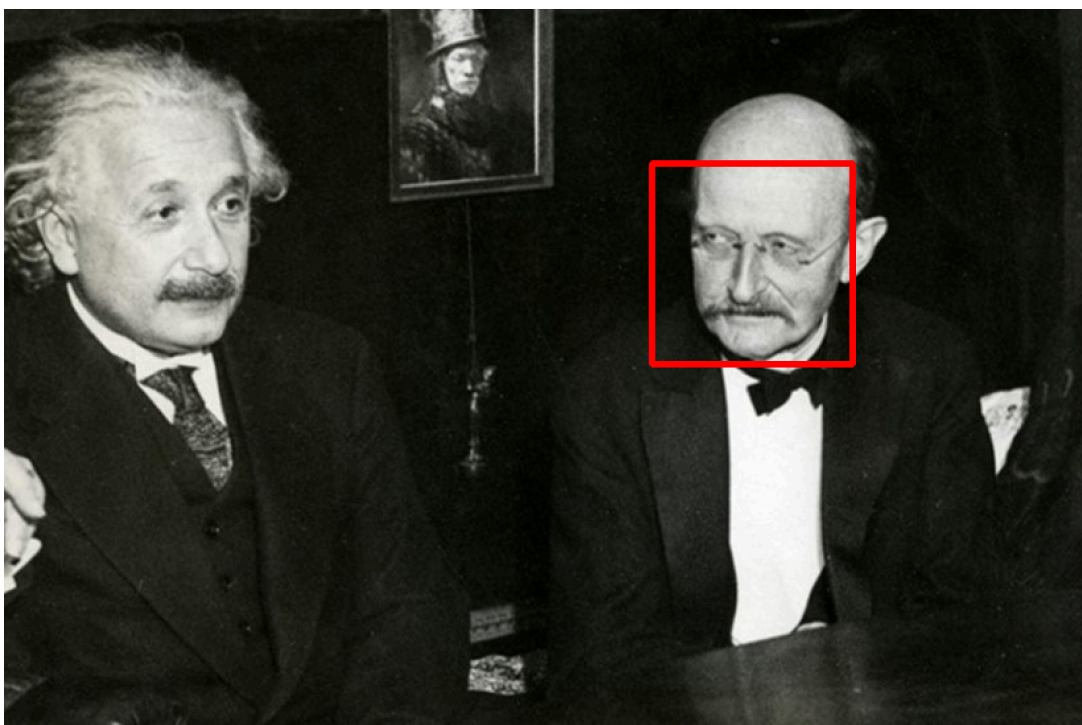
#Buscamos los rostros:
rostros = modelo.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6 )

#Ahora recorremos el array 'rostros' y dibujamos los rectángulos sobre la imagen original:
for (x,y,ancho, alto) in rostros:
    cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)

#Abrimos una ventana con el resultado:
cv2.imshow('Output', img)
print("\nMostrando resultado. Pulsa cualquier tecla para salir.\n")
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Como hemos visto en prácticas empleamos la librería de opencv para realizar el reconocimiento de las imágenes. En este caso cargo el modelo clasificador frontalface_alt de Haar con la función CascadeClassifier de opencv y tras cargar la imagen busco los rostros en la imagen y muestro por pantalla los resultados

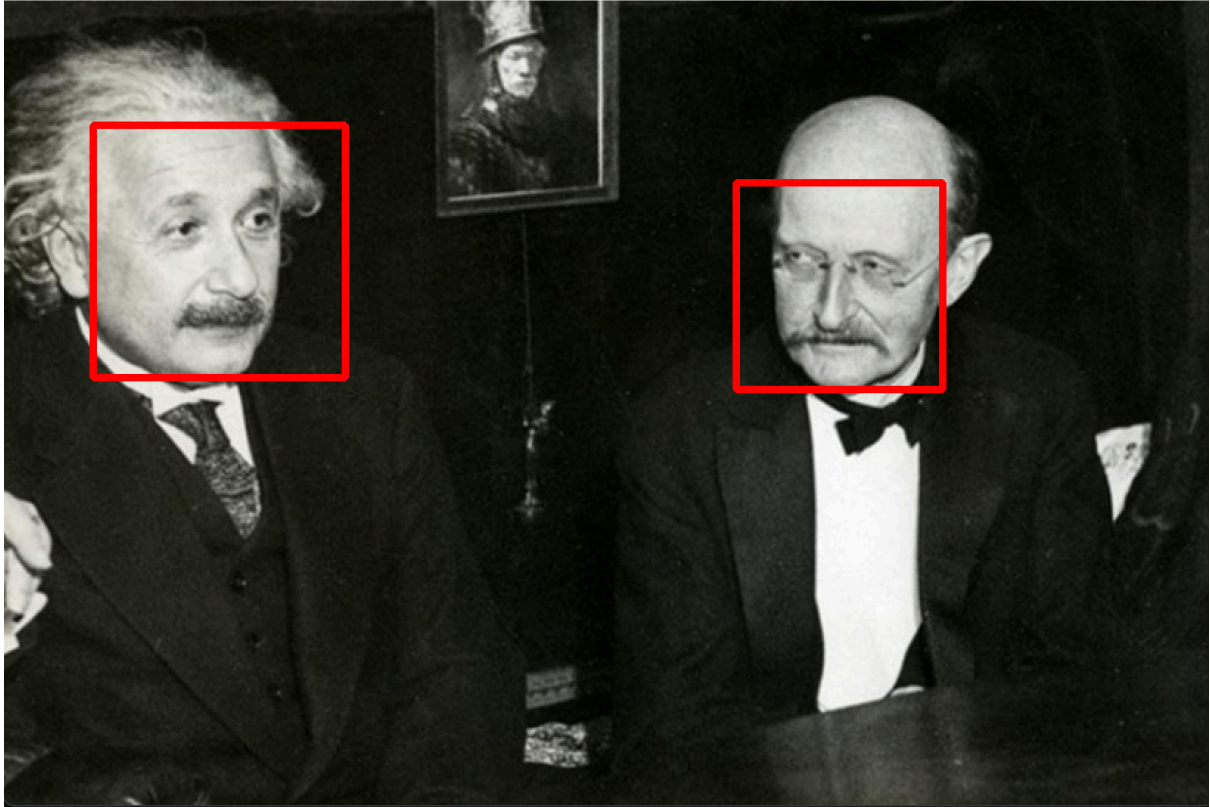
Resultados:



En este caso he empleado un factor de escala de 1.5, cuanto más cercano a 1 sea el factor, más tardará el programa en detectar una solución, aunque menos probable será que se le escapen algunas caras; y un valor de detecciones de 6, a mayor sea este valor, menos detecciones se realizarán, aunque mayor calidad tendrán estas.

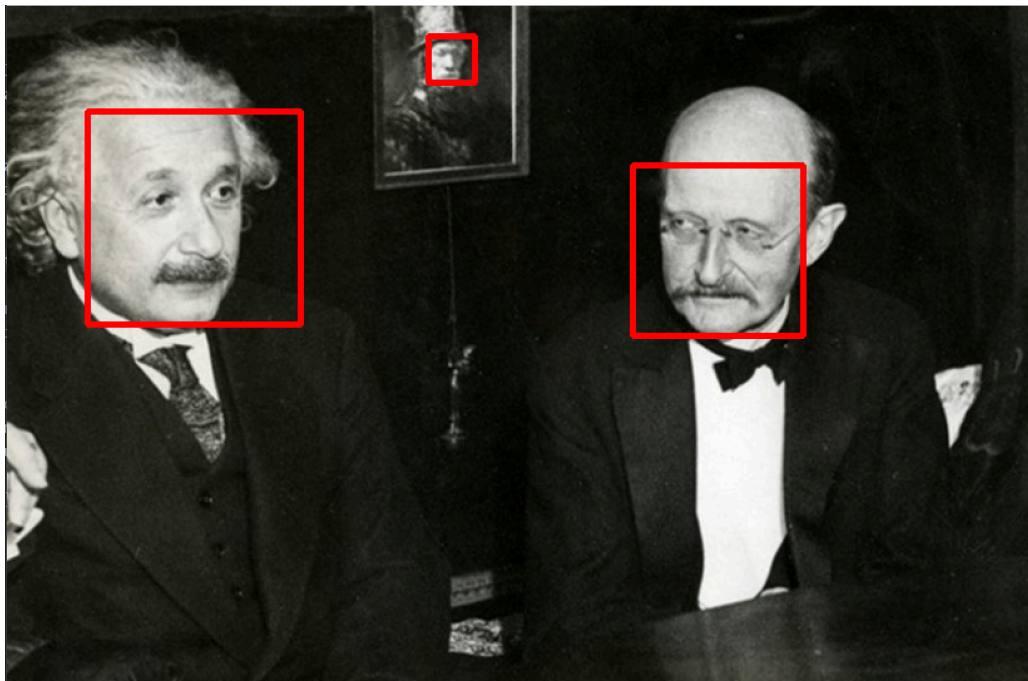
Podemos observar que en este caso solo se ha detectado una de las tres caras posibles, teniendo en cuenta también la cara del cuadro que hay en segundo plano.

Segundo intento:



Para este segundo intento, he reducido el scale factor a 1.2. Ahora observamos que si que detecta la segunda cara que aparece en la imagen, aunque sigue sin detectar la imagen del cuadro que se encuentra en segundo plano.

Tercer intento:



Para este último intento comencé intentando disminuir el valor de detecciones a la mitad, aunque sin éxito, puesto que seguía sin detectarse el rostro del retrato del cuadro. Sin embargo, al volver a disminuir el factor de escala a 1.1 y como podemos observar en el resultado, finalmente conseguimos que el programa detecte todas las caras de la imagen.

Ejercicio 2: Caras de personas en vídeos

Para este segundo ejercicio se nos pide que detectemos los rostros de personas en vídeos, marcando la zona que corresponde la de la cara de las personas que aparezcan en el vídeo.

Código:

```
import cv2

ruta1 = r'..\modelos\haarcascade_frontalface_default.xml'
ruta2 = r'..\videos\v2.mp4'
# Cargamos el modelo clasificador para la cara
modeloCara = cv2.CascadeClassifier(ruta1)

#Cargamos un vídeo:
camara = cv2.VideoCapture(ruta2)

while camara.isOpened():
    # Cargamos en img lo capturado por la cámara
    _, img = camara.read()

    color = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)

    # Detectamos la cara
    faces = modeloCara.detectMultiScale(color, 1.1, 4)

    # Pintamos el rectángulo sobre la cara
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x,y), (x+w, y+h), (255, 0, 0), 3)

    # Lo mostramos en pantalla mientras no pulsemos la tecla 'esc'
    cv2.imshow('img', img)
    key = cv2.waitKey(1)
    if key == 27:
        break

camara.release()
cv2.destroyAllWindows()
```


Para este segundo ejercicio empleo el mismo modelo que en el ejercicio anterior, aunque en este caso detectamos los rostros mientras el vídeo siga corriendo, en lugar de sobre una imagen en estático como en el ejercicio anterior.

Resultado:



En este caso he realizado las pruebas con el siguiente vídeo en el que se ve a dos personas andando, y a lo largo de todo el vídeo se puede observar que se marcan constantemente los rostros de estas personas a lo largo del vídeo, aunque se vayan desplazando y vayan moviendo la cabeza.

Ejercicio 3: Caras de gatos en imágenes

Para este tercer ejercicio vamos a detectar en este caso los rostros de gatos en lugar de personas humanas, marcando las zonas de la imagen donde se detecten las caras de estos felinos.

Código:

```
import cv2

modelo = cv2.CascadeClassifier('./modelos\haarcascade_frontalcatface_extended.xml')

img = cv2.imread('./imagenes\imagen_gatos.jpg')
imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

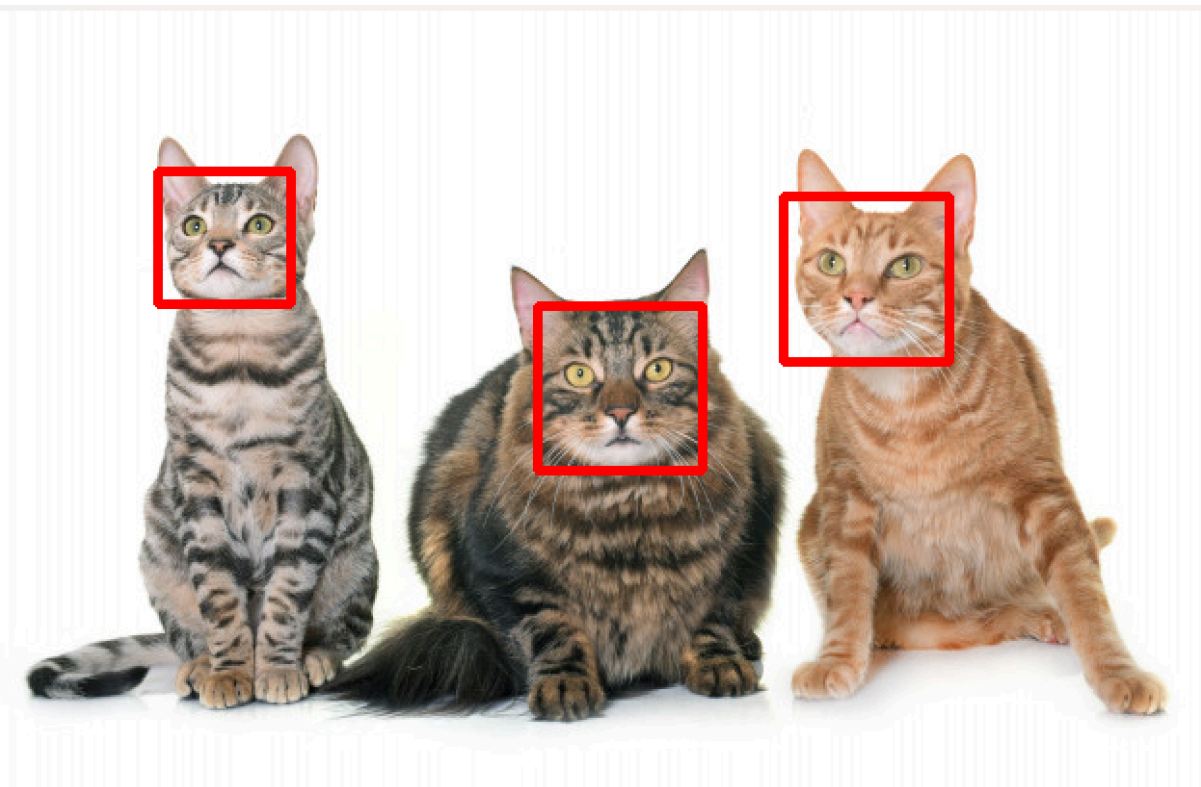
gatos = modelo.detectMultiScale(imagen, scaleFactor=1.1, minNeighbors=6 )

for (x,y,ancho, alto) in gatos:
    cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)

cv2.imshow('Output', img)
print("\nMostrando resultado. Pulsa cualquier tecla para salir.\n")
cv2.waitKey(0)
cv2.destroyAllWindows()
```

En este caso empleamos exactamente el mismo procedimiento que para la detección de rostros humanos en imágenes, aunque en este caso modificando el modelo por uno entrenado por rostros de gatos.

Resultado:



En este caso he empleado un factor de escala de 1.1 y he mantenido el valor de detección a 6, lo que ha resultado suficiente para poder detectar las caras de todos los gatos de la imagen como se puede observar en el resultado.

Ejercicio 4: Cuerpos y Caras de personas en vídeos

Para este cuarto y último ejercicio, la idea es lograr detectar el cuerpo y la cara de personas en vídeos de manera simultánea, marcando las zonas donde se encuentren los cuerpos de las personas y las zonas donde se encuentren sus rostros a la misma vez.

Código:

```
import cv2

modeloCara = cv2.CascadeClassifier(r'.\modelos\haarcascade_frontalface_default.xml')

modeloCuerpo = cv2.CascadeClassifier(r'.\modelos\haarcascade_fullbody.xml')

camara = cv2.VideoCapture(r'.\videos\v4.mp4')

while camara.isOpened():

    _, img = camara.read()

    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    caras = modeloCara.detectMultiScale(gris, 1.1, 6)

    cuerpos = modeloCuerpo.detectMultiScale(gris, 1.1, 3)

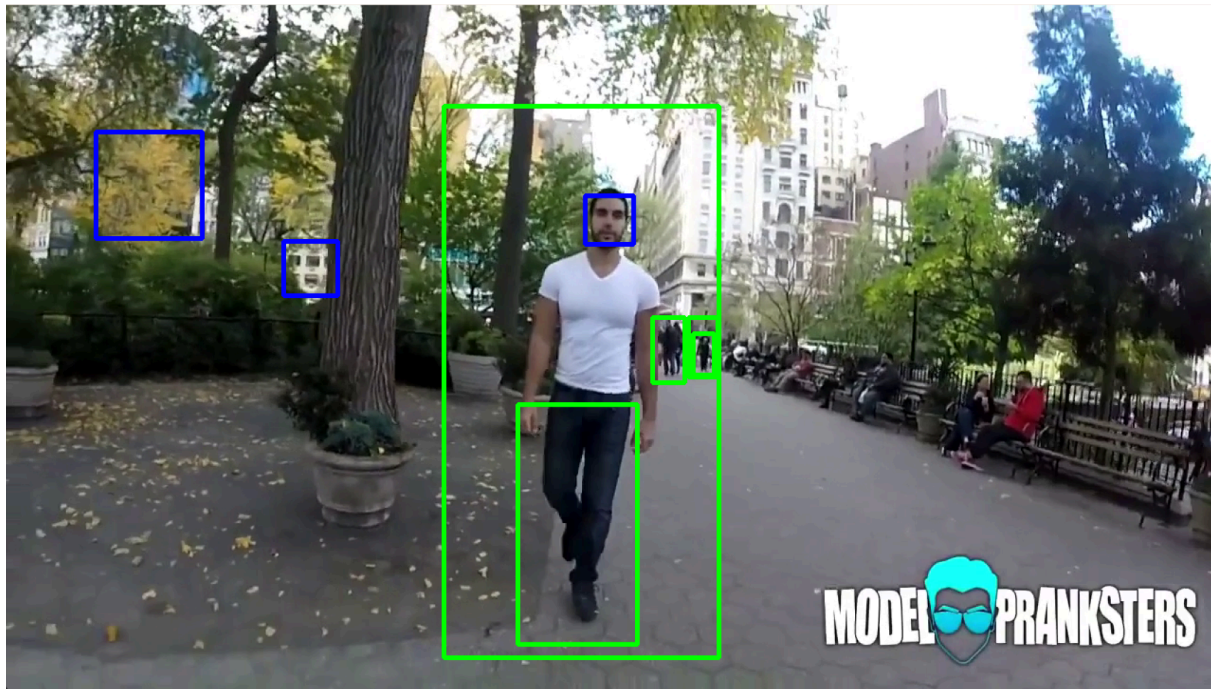
    for (x, y, w, h) in caras:
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 3)

    for (x, y, w, h) in cuerpos:
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 3)

    cv2.imshow("Deteccion", img)
    key = cv2.waitKey(1)
    if key == 27:
        break

camara.release()
cv2.destroyAllWindows()
```

Resultado:



Como se puede observar en el resultado, el modelo funciona, puesto que detecta bien algunas caras y cuerpos, aunque no al 100%, puesto que hay algunos cuerpos que no detecta como el de la persona que está en el banco en ese frame concreto, o porque en algunos casos detecta cosas que no son, como por ejemplo que detecta caras en el fondo que no son, o detecta un cuerpo extra en los pies de la persona que podemos observar al frente de la imagen.